

Nukkit 脚本扩展：目录插件

本文说明 HuHoBotPenguin 的 Nukkit-MOT 适配器如何加载脚本插件。一个插件是 addons/ 下的一个目录，不是单个文件。

- 适用版本：1.14.0
- 插件名：HuHoBotPenguin-NukkitPlatform（命令别名 /hb）
- 加载器：server/Nukkit/src/main/java/cn/huohuas001/huhobotPenguin/nukkit/scripting/
- Lua 约定来自 NuclearScripting（Shiiyuko，2024）
- JavaScript 引擎与 AXDA-ScriptEngine（MIT）同款 GraalJS，但只暴露 mc，不是整套 LSE API
- Python 用 GraalPy 24.1（Python 3），桥接类 cn.huohuas001.huhobot.graalpy.GraalPyBridge，与 Spigot 共用同一个引擎 jar

插件启动时只扫描自己的数据目录 addons/，不会去扫服务器的 plugins/。addons/ 根上直接放 .lua / .js / .py 不再加载，控制台会提示把它挪进目录。

1. 一个插件长什么样

```
plugins/HuHoBotPenguin-NukkitPlatform/addons/welcome/
├── metadata.yaml      名字、版本、作者、描述、入口文件
├── _conf_schema.json  配置项和默认值（可省略）
├── requirements.txt   依赖声明（只预检，不安装，可省略）
└── main.lua          入口。也可以是 main.py 或 main.js
```

metadata.yaml 只认顶层的 key: value：

```
name: welcome
version: 1.0.0
author: HuHoBot
description: 进服欢迎
entry: main.lua      # 可省略
```

缺省时名字和作者用目录名，版本是 1.0.0。entry 没写就按 main.lua、main.py、main.js 找第一个存在的文件。三种都没有，这个目录被跳过。

脚本里还能用 PLUGIN_VERSION、PLUGIN_AUTHOR、PLUGIN_DESCRIPTION 三个全局变量覆盖 metadata（Lua 和 Python）。

配置和代码是分开的，更新插件不会丢用户改过的值：

路径	是什么
addons/<插件>/_conf_schema.json	配置声明，每个键一个 {type, description, default}
addons/config/<插件名>.json	实际配置。首次加载按 schema 的 default 生成，之后只补缺的键
addons/data/<插件名>/	大文件目录，脚本里叫 DATA_DIR
addons/data/<插件名>/kv.properties	键值存储，脚本里叫 kv

schema 例子：

```
{
  "token": {"type": "string", "description": "访问令牌", "default": ""},
  "groups": {"type": "list", "description": "互通群号", "default": []}
}
```

addons/config/<插件名>.json 里写 `"_enabled": false` 就禁用这个插件，加载时跳过。

requirements.txt 只做预检。Python 会逐个 import，缺了记一条警告；Lua 和 JavaScript 没有包管理器，只把声明的依赖打印出来。三种语言都不会自动安装。

三种语言都注入同一组存储对象：

名字	是什么	Lua	JavaScript / Python
config	schema 驱动的配置	config.get("token")	config.get("token")
kv	字符串键值，写入即落盘	kv.put("k", "v")	kv.put("k", "v")
DATA_DIR	数据目录的绝对路径	字符串	字符串

config 的方法：`get(key)`、`get(key, 默认值)`、`getString(key)`、`getString(key, 默认值)`、`getList(key)`（元素都是字符串）、`set(key, 值)`、`remove(key)`、`all()`。kv 的方法：`get(key)`、`get(key, 默认值)`、`put(key, 值)`、`delete(key)`、`all()`。

2. addon 模块

settings.gradle.kts 里的脚本引擎都不进主插件 jar。Nukkit 用下面两个；这两个引擎包 Nukkit 和 Spigot 共用，放进各自插件的 engines/ 即可。

模块	是什么	产物与位置
:addon-GraalJs	GraalJS 24.1.2，与 Spigot 共用	HuHoBot-Engine-GraalJs-<版本>.jar，放到 engines/
:addon-GraalPy	GraalPy 24.1.2（Python 3），与 Spigot 共用	HuHoBot-Engine-GraalPy-<版本>.jar，放到 engines/
主 jar	Lua 5.4（LuaJava）	已打进 HuHoBot-Penguin-Nukkit-<版本>.jar

```
./gradlew :addon-GraalJs:shadowJar :addon-GraalPy:shadowJar
# addon/GraalJs/build/libs/HuHoBot-Engine-GraalJs-<版本>.jar
# addon/GraalPy/build/libs/HuHoBot-Engine-GraalPy-<版本>.jar
```

engines/ 在插件启动时创建。第一次加载 .js 或 .py 时，NukkitScriptLoader 把该目录里所有 *.jar 挂到同一个 URLClassLoader（父加载器是插件自己的），之后缓存复用。

主插件编译期不依赖 Graal。对引擎的调用全部走反射（cn.huohuas001.huhobot.graaljs.GraalJsBridge、cn.huohuas001.huhobot.graalpy.GraalPyBridge），所以缺引擎 jar 时主 jar 依然能编译、能启动，只是对应语言的本报「未安装脚本引擎」。

入口	引擎与位置	脚本怎么写
main.lua	Lua 5.4 (LuaJava)，在主 jar	全局函数 onPlayerJoin(event) 即监听 PlayerJoinEvent
main.js	GraalJS 24.1.2, engines/HuHoBot-Engine-GraalJs-*.jar	顶层调用 mc.listen(...)
main.py	GraalPy 24.1.2 (Python 3), engines/HuHoBot-Engine-GraalPy-*.jar	on_enable() 里调用 api.register_event(...)

重载按**目录名**，大小写不敏感：

```
/huhobot scripts reload
/huhobot scripts reload welcome
```

权限是 huhobot.command，默认 OP。别名 /hb 同样可用。

3. 加载框架

```
plugins/HuHoBotPenguin-NukkitPlatform/
├── addons/
│   ├── welcome/
│   │   ├── metadata.yaml
│   │   ├── _conf_schema.json
│   │   └── main.lua
│   ├── config/                各插件的实际配置，与代码分开
│   │   └── welcome.json
│   └── data/
│       └── welcome/           DATA_DIR, 里面还有 kv.properties
└── engines/
    ├── HuHoBot-Engine-GraalJs-<版本>.jar
    └── HuHoBot-Engine-GraalPy-<版本>.jar

HuHoBotNukkit.onEnable()
    |
    v
NukkitScriptLoader.loadAll()
只看 addons/ 下的目录，跳过 config/ 和 data/
    |
    +-- 读 metadata.yaml, 找入口
    +-- 打开 config (schema 默认值补齐) 和 kv
    +-- _enabled=false 就跳过
    |
    +-- main.lua --> Lua54, 主 jar 就能跑
    +-- main.js --> engines/ 有 GraalJS jar ? 没有则这条失败
    +-- main.py --> engines/ 有 GraalPy jar ? 没有则这条失败
    |
```

v

AddonManager.register(metadata 里的名字)

卸载 (/huhobot scripts reload, 或插件 onDisable) 按语言分别做:

语言	卸载时
Lua	若有 onDisable(instance) 就调用; 删掉 botCommands:register 登记的 QQ 命令; lua.close() 释放原生状态
JavaScript	取消 setTimeout / setInterval; 用 CommandMap.unregister 删掉 mc.registerCommand 登记的命令; 删掉 QQ 命令; 关闭 GraalJS 上下文
Python	若有 on_disable() 就调用; 取消定时任务、Nukkit 命令和 QQ 命令; 关闭 GraalPy 上下文

一个目录失败只记一条日志, 不登记为 addon, 其它插件继续。

4. Lua

源码: LuaScriptEngine。引擎打在主 jar 里, 不需要 engines/。

4.1 加载

- new Lua54(), openLibraries() 打开标准库和 java 库。
- 注入全局对象:

| 名字 | 是什么 | |-----|-----| | plugin | HuHoBotNukkit 插件实例 | | server | Nukkit Server | | logger | 插件日志 | | SCRIPT_NAME | metadata 里的名字 | | config / kv / DATA_DIR | 见第 1 节 | | botCommands | 只含 register(key, 命令模板) 的桥, 用来登记 QQ 命令 |

- lua.run 执行入口文件的顶层。
- 用 metadata 登记 addon。PLUGIN_VERSION、PLUGIN_AUTHOR、PLUGIN_DESCRIPTION 若存在则覆盖。描述为空时记为 Lua 5.4 script addon。
- bindEvents(): 遍历 _G, 见下节。
- 若存在函数 onEnable, 调用 onEnable(plugin)。

4.2 事件名

NuclearScripting 的约定: 完整事件名前加 on, 删掉末尾的 Event。

想监听的事件	写成函数
PlayerJoinEvent	function onPlayerJoin(event)
PlayerChatEvent	function onPlayerChat(event)
PlayerQuitEvent	function onPlayerQuit(event)
BlockBreakEvent	function onBlockBreak(event)

加载器拿函数名去掉 on 再补回 Event, 然后按顺序在这些包里找类:

```
cn.nukkit.event.player.  
cn.nukkit.event.block.  
cn.nukkit.event.entity.
```

```
cn.nukkit.event.inventory.  
cn.nukkit.event.level.  
cn.nukkit.event.server.  
cn.nukkit.event.plugin.  
cn.nukkit.event.
```

找到就用 `PluginManager.registerEvent` 注册，优先级 `NORMAL`，不是 `ignore-cancelled`。找不到就跳过，不报错。
`onEnable` 与 `onDisable` 被明确排除。回调里抛出的错误会被捕获，写一条警告。

4.3 与 Java 互操作

LuaJava 在 Lua 侧提供 `java` 模块（<<https://luajava.iiroiro.party/api.html>>）：

```
local Integer = java.import("java.lang.Integer")  
local n = Integer.parseInt("7")
```

- `java.import("完整类名")` 返回一个类。
- 静态方法和实例方法用冒号调用：`player.sendMessage("hi")`。
- 读字段用点：`player.name`。
- Java 数组在 Lua 侧仍从 1 开始：`array[1]` 是 Java 的 `array[0]`。
- 构造对象用 `java.new(类, 参数...)`。

4.4 示例

目录：`plugins/HuHoBotPenguin-NukkitPlatform/addons/welcome/`，入口 `main.lua`。

```
function onEnable(instance)  
    instance.getLogger():info("welcome 已加载，令牌长度 " .. #config.getString("token"))  
    botCommands.register("欢迎", "say {name} 说: {params}")  
end  
  
function onPlayerJoin(event)  
    local player = event.getPlayer()  
    player.sendMessage("欢迎, " .. player.getName())  
    kv.put("last_join", player.getName())  
end  
  
function onDisable(instance)  
    instance.getLogger():info("welcome 已卸载")  
end
```

`botCommands.register(key, 命令模板)` 登记的是 QQ 群自定义命令，权限公开，并推送到 QQ 菜单。命令模板是一条服务器命令，占位符与配置文件里的自定义命令相同：`{params}`、`{group}`、`{user}`、`{name}`、`{nickname}`、`{0}`、`{1}`。脚本重载时这些 `key` 会被删除。

没有 `onEnable` 或 `onDisable` 完全合法。事件函数在 `onEnable` 之前就已经绑定好。

5. JavaScript

源码：JsScriptEngine。引擎来自 :addon-GraalJs（与 Spigot 共用），桥接类是 cn.huohuas001.huohobot.graaljs.GraalJsBridge。GraalVM 24.1 从当前线程的 context classloader 发现语言，而且 Context.newBuilder 没有 classloader 参数，所以上下文必须在引擎 jar 自己的加载器里创建。主插件只通过反射调用 GraalJsBridge.open()。

Nukkit 这份桥**没有**打开 Nashorn 兼容，脚本里没有 Java.type。需要 Java 对象时，用注入进来的 mc、server、plugin、config、kv，以及它们返回的 Nukkit 对象。

5.1 加载

每个插件一个独立上下文，全局变量互不可见。注入：

名字	是什么
mc	JsApi，事件、命令、定时器、QQ 命令
server	Nukkit Server
plugin	HuHoBotNukkit
logger	插件日志
config / kv / DATA_DIR	见第 1 节

入口文件的顶层在加载时执行，没有 onEnable。成功后按 metadata 登记 addon，描述为空时记为 JavaScript script addon (GraalJS)。语法或运行错误记一条 [名字] JavaScript 错误：...，这个插件不登记。

5.2 mc 桥

方法	作用
mc.addonName()	metadata 里的名字
mc.log(文本) / mc.warn(文本)	写日志，前缀是插件名
mc.broadcast(文本)	广播
mc.sendBotText(文本)	把文本发到已配置的全部 QQ 群
mc.getPlayer(名字)	找在线玩家，没有返回 null
mc.tell(玩家, 文本)	给一个玩家发消息
mc.runCommand(命令)	下一拍以控制台身份执行，自动去掉开头的 /
mc.getServer()	Nukkit Server
mc.listen(事件名, 函数)	监听。事件名可用简单名 PlayerJoinEvent，也可用全限定名。回调签名 (event)
mc.registerCommand(名字, 函数)	注册 Nukkit 命令，回调签名 (sender, label, args)
mc.registerBotCommand(key, 命令模板)	登记 QQ 群命令，权限公开，并推送到菜单
mc.registerBotCommand(key, 命令模板, 权限)	权限大于 0 仅管理员
mc.setTimeout(函数, 延迟拍)	延迟执行，20 拍 = 1 秒，返回 taskId
mc.setInterval(函数, 周期拍)	周期执行，返回 taskId

方法	作用
<code>mc.clearInterval(taskId)</code>	取消

`listen` 的简单名按 `ScriptNames.findEventClass` 解析，包顺序与 `Lua` 相同。找不到、或第二个参数不是函数，只记一条警告。监听器优先级 `NORMAL`，被其它插件取消的事件仍会送到脚本。

脚本卸载时，它登记的 `Nukkit` 命令、`QQ` 命令和定时任务会一起取消。

5.3 示例

入口：`addons/welcome/main.js`。需要 `engines/` 里的 `GraalJS jar`。

```
mc.registerBotCommand("欢迎", "say {name} 说: {params}");

mc.listen("PlayerJoinEvent", function (event) {
    var player = event.getPlayer();
    mc.tell(player, "欢迎, " + player.getName());
    kv.put("last_join", player.getName());
});

mc.registerCommand("hello", function (sender, label, args) {
    sender.sendMessage("你好, " + sender.getName());
});
```

6. Python

源码：`GpyScriptEngine`。引擎来自 `:addon-GraalPy`（与 `Spigot` 共用），是 `GraalPy 24.1`，**Python 3**。上下文同样必须建在引擎 `jar` 自己的 `classloader` 上，主插件只反射调用 `GraalPyBridge.open()`。

每个插件一个独立上下文。注入：

名字	是什么
<code>api</code>	事件、命令、定时器、 <code>QQ</code> 命令
<code>server</code>	<code>Nukkit Server</code>
<code>plugin</code>	<code>HuHoBotNukkit</code>
<code>logger</code>	插件日志
<code>SCRIPT_NAME</code>	<code>metadata</code> 里的名字
<code>config/kv/DATA_DIR</code>	见第 1 节

执行入口文件后，用 `metadata` 登记 `addon`，`PLUGIN_VERSION` 等三个变量可以覆盖。描述为空时记为 `Python script addon (GraalPy)`。若定义了 `on_enable` 就调用它。

6.1 api 桥

方法	作用
<code>api.addon_name()</code>	metadata 里的名字
<code>api.log(文本) / api.warn(文本)</code>	写日志，前缀是插件名
<code>api.broadcast(文本)</code>	广播
<code>api.send_bot_text(文本)</code>	把文本发到已配置的全部 QQ 群
<code>api.get_player(名字)</code>	找在线玩家，没有返回 None
<code>api.tell(玩家, 文本)</code>	给一个玩家发消息
<code>api.dispatch_command(命令)</code>	当前线程 以控制台身份执行，自动去掉开头的 /
<code>api.register_event(事件名, 函数)</code>	监听。事件名可用简单名，也可用全限定名。回调签名 (event)
<code>api.register_command(名字, 函数)</code>	注册 Nukkit 命令，回调签名 (sender, label, args)
<code>api.register_command(名字, 描述, 函数)</code>	同上，多一个描述
<code>api.register_bot_command(key, 命令模板)</code>	登记 QQ 群命令，权限公开
<code>api.register_bot_command(key, 命令模板, 权限)</code>	权限大于 0 仅管理员
<code>api.schedule_task(函数, 延迟拍)</code>	延迟执行，20 拍 = 1 秒
<code>api.schedule_repeating_task(函数, 周期拍)</code>	周期执行
<code>api.cancel_task(taskId)</code>	取消

事件名解析与 JavaScript 相同。脚本卸载时，它登记的 Nukkit 命令、QQ 命令和定时任务会一起取消。

GraalPy 默认不带第三方库。requirements.txt 里声明的模块在加载前会 import 一次，缺了只记警告。

6.2 示例

入口：addons/welcome/main.py。需要 engines/ 里的 GraalPy jar。

```
def on_join(event):
    player = event.getPlayer()
    api.tell(player, "欢迎, " + player.getName())
    kv.put("last_join", player.getName())

def on_hello(sender, label, args):
    sender.sendMessage("你好, " + sender.getName())

def on_enable():
    api.log("welcome 已加载")
    api.register_event("PlayerJoinEvent", on_join)
    api.register_command("hello", "打个招呼", on_hello)
    api.register_bot_command("欢迎", "say {name} 说: {params}")

def on_disable():
    api.log("welcome 已卸载")
```

7. 三种写法对照

目的	Lua 5.4	JavaScript	Python 3
加载钩子	<code>function onEnable(instance)</code>	没有，顶层即执行	<code>def on_enable():</code>
卸载钩子	<code>function onDisable(instance)</code>	没有	<code>def on_disable():</code>
监听进服	<code>function onPlayerJoin(event)</code>	<code>mc.listen("PlayerJoinEvent", fn)</code>	<code>api.register_event("PlayerJoinEvent", on_join)</code>
游戏内命令	自行 <code>java.import</code> 后注册	<code>mc.registerCommand("hello", fn)</code>	<code>api.register_command("hello", fn)</code>
QQ 群命令	<code>botCommands.register(key, 模板)</code>	<code>mc.registerBotCommand(key, 模 板)</code>	<code>api.register_bot_command(key, 模板)</code>
发到全部 QQ 群	没有专用桥	<code>mc.sendBotText(文本)</code>	<code>api.send_bot_text(文本)</code>
读配置	<code>config.getString("token")</code>	<code>config.getString("token")</code>	<code>config.getString("token")</code>
调用 Java 方法	冒号： <code>player.sendMessage("hi")</code>	点： <code>player.getName()</code>	点： <code>player.getName()</code>
引擎位置	主 jar	<code>engines/HuHoBot-Engine- GraalJs-*.jar</code>	<code>engines/HuHoBot-Engine-GraalPy-*.jar</code>

8. 安全边界

Lua 打开了全部标准库，`io`、`os` 可用，`java.import` 能加载任意类。JavaScript 和 Python 的上下文都是 `allowAllAccess(true)`，能碰到注入进来的全部 Java 对象。`api.dispatch_command` 以控制台身份执行命令。

这是有意的：脚本作者就是服务器管理员。不要把 `addons/` 交给不信任的人。`botCommands / mc / api` 只是窄桥，限制不了 `java.import("java.io.File")`。